

The '::' symbol is used to separate the expression on the left with the data type on the right.

A *Bool* data type represents a logical value of either *True* or *False*:

```
Prelude> :type True
True :: Bool
```

Signed numbers with a fixed width are represented by the *Int* data type. The *Integer* type is used for signed numbers that do not have a fixed width:

```
Prelude> 5
5
```

The *Double* and *Float* types are used to represent decimal values. The *Double* type has better precision for floating point numbers:

```
Prelude> 3.8
3.8
```

The basic data types can be combined to form composite types. There are two widely used composite types in Haskell, namely, lists and tuples. A list is a collection of elements of the same data type enclosed within square parentheses. A list of characters is shown below:

```
Prelude> :type ['a', 'b', 'c']
['a', 'b', 'c'] :: [Char]
```

The static typing in Haskell produces errors during compile or load time (in GHCi) when you mix data types inside a list. For example:

```
Prelude> ['a', 1, 2]
```

```
<interactive>:7:7:
```

```
No instance for (Num Char) arising from the literal '1'
Possible fix: add an instance declaration for (Num Char)
In the expression: 1
In the expression: ['a', 1, 2]
In an equation for 'it': it = ['a', 1, 2]
```

You can have a list of lists as long as it contains the same data type:

```
Prelude> :type [['a'], ['b', 'c']]
[['a'], ['b', 'c']] :: [[Char]]
```

A tuple is an ordered list of elements with a fixed size, enclosed within parentheses, where each element can be of a different data type. For example:

```
Prelude> :type ('t', True)
('t', True) :: (Char, Bool)
```

Note that the tuple with type (Char, Bool) is different from the tuple with type (Bool, Char):

```
Prelude> :t (False, 'f')
(False, 'f') :: (Bool, Char)
```

Haskell originates from the theory of Lambda calculus, which was developed by Alonzo Church to formally study mathematics. In 1958, John McCarthy created Lisp, which relates programming with Lambda calculus. Robin Milner created a functional programming language called ML (meta language) for automated proofs of mathematical theorems in 1970. During the 1980s, there were a number of lazy functional programming languages scattered across the research community. Miranda was a very popular proprietary programming language released by Research Software Ltd in 1985.

A need arose to unify the different research developments, for which a committee was formed and the first version of the standard was released in 1990. It was called Haskell 1.0, after the mathematician and logician, Haskell Brooks Curry. Subsequently, there were four revisions made - 1.1, 1.2, 1.3 and 1.4. In 1997, the Haskell 98 report was released. In 2009, the Haskell 2010 standard was published and is the latest standard, as on date. It has Foreign Function Interface (FFI) bindings to interface with other programming languages. The Hugs interpreter is useful for teaching, while the Glasgow Haskell Compiler (GHC) is very popular. The paper by John Hughes on 'Why Functional Programming Matters?' is an excellent paper to read. A number of software companies in the industry have begun using Haskell in production systems.

We shall be exploring more features, constructs and uses of the language in future articles. **END** 

References

- [1] Haskell. <http://haskell.org/>
- [2] Haskell 2010. http://www.haskell.org/haskellwiki/Haskell_2010
- [3] Hoogse. <http://www.haskell.org/hoogse/>
- [4] Hayoo. <http://holumbus.fh-wedel.de/hayoo/hayoo.html>
- [5] Hackage. <http://hackage.haskell.org/>
- [6] Haskell. Platform. <http://www.haskell.org/platform/>
- [7] Glasgow Haskell Compiler. <http://www.haskell.org/ghc/>

By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com